Draft version October 7, 2026
Typeset using LaTeX **modern** style in AASTeX7.0.1

salvador

Deimer Antonio Alvarez Garay [1]

[1] *INAF - Osservatorio Astrofisico di Arcetri, Largo E. Fermi 5, 50125 Firenze, Italy*

1. INTRODUCTION

`salvador` is a `Python` code designed to perform chemical abundance determination via spectral synthesis. In particular, `salvador` performs a χ^2 minimization between an observed line and a grid of suitable synthetic spectra calculated using the code `SYNTHE` (R. L. Kurucz 2005) in which only the abundance of the element under analysis is varied. The `SYNTHE` code is managed by another `Python` code, called `K2` written by Alessio Mucciarelli (see the K2 documentation for the installation and usage of `K2`).

Unlike other codes in the literature (e.g.; L. Magrini et al. 2013; L. Sbordone et al. 2014; S. Blanco-Cuaresma et al. 2014), `salvador` is designed exclusively for the determination of chemical abundances (and, for specific lines, of the projected rotational velocity, see Sect. 8). Consequently, the atmospheric parameters of the stars under analysis must be known beforehand, as their determination is not included in the code.

2. INSTALLATION

Once that you have installed `K2` in your laptop you can install and use `salvador`. To do that, download the necessary files from XXXX:

xxx

Unzip the **salvador.zip** into a directory named **salvador**[2] by typing from your terminal:

unzip salvador.zip

The directory **salvador** will contain the following files and directories:

- salvador.py: the program that you run.
- utilities_salvador.py: a short file that makes all the functions of `salvador` available to salvador.py.

Email: deimer.alvarez@inaf.it

[2] I suggest to create a directory **software** in your home and to unzip `salvador` there.

- The directory named **salvador_lib**: it contains all the functions of `salvador`, divided into modules according to their task (see Appendix C).
- solar_abu.txt (see the following for the explanation)
- cows.py (see Appendix A for the explanation)
- merge_fit.py (see Appendix B for the explanation)
- regression_check.py and check_k2_speedup.py: two tools to check the results of `salvador` (see Sect. 10).
- Makefile and requirements.txt: to install `salvador` and the `Python` packages that it needs.
- FUNCTION_MAP.txt: the list of all the functions of `salvador`, with the module where each one is and a short description.
- The directory named **tutorial**

In order to run `salvador` the scripts must be executable and the path of the **salvador** directory must be saved in your login file (the most common are .bashrc, .tcshrc, and .zshrc). All these instructions are included into the **Makefile**. Therefore, once that you have unpacked the .zip file go into the **salvador** directory and from terminal type:

make install

and then, to apply the change to the terminal that is already open (or simply open a new terminal):

source ~/.bashrc

using the name of your login file. The command **make install** makes the scripts executable, adds the path of the **salvador** directory to your login file (only once, also if you run it again), and checks the installation. The other commands available are:

- **make check**: checks that everything needed by `salvador` is in place: all its files (including **solar_abu.txt**), the scripts executable, `Python` 3.9 or newer, the `Python` packages (see Sect. 2.1), `K2` (the command **k2.py** must be found), and the path in your login file. Every line should end with [OK]; for each [FAIL] the reason is printed (for a missing package also the command to install it). You can run it at any time.
- **make clean**: removes the `Python` caches (the **__pycache__** directories).
- **make uninstall**: removes the path of the **salvador** directory from your login file. A copy of the file before the change is saved with the extension .bak.
- **make help**: lists the commands available.

If for some reason you get some errors, you can make the scripts executable by typing from terminal into the salvador directory:

chmod +x salvador.py regression_check.py check_k2_speedup.py
chmod +x cows.py merge_fit.py

Then put the path of directory **salvador** in your login file according to the shell environment of your machine.

For example, if your configuration file is a .bashrc, open it and put the following command: **export PATH=/path/to/salvador:$PATH**

To update `salvador`, copy the new files over the old ones: there is no need to install it again. If you move the **salvador** directory, run **make uninstall** in the old place and **make install** in the new one.

`salvador` is always run from the directory of your data (the input files and the spectra, see Sect. 3), where also all the output files are written (see Sect. 4): the **salvador** directory contains only the code.

Check the correct installation of `salvador` by running `salvador.py` into the **salvador/tutorial/** directory.

The **solar_abu.txt** is a file that is read by `salvador` and serves to set the solar abundance reference for the calculation of the final abundance in the form [Fe/H] or [X/Fe]. To the date the file contains 5 columns (see Fig. 1): the first contains the atomic number of the element, the second the corresponding symbol, while the other 3 columns contain the solar abundances from E. Caffau et al. (2011) and K. Lodders (2010), N. Grevesse & A. J. Sauval (1998), and E. Magg et al. (2022). Other columns can be added and used as a solar reference. This file must be located in the same directory in which is located **salvador.py** (the **salvador** directory).

The default reference used by `salvador` is E. Caffau et al. (2011) and K. Lodders (2010), but it can be changed in the input.salvador file (see 3.1 for the explanation).

#Elem	code	CL	GS98	Magg22	Asplund09
1	H	12.00	12.00	12.00	12.00
2	He	10.94	10.93	10.93	10.93
3	Li	1.03	1.10	1.05	1.05
4	Be	1.32	1.40	1.38	1.38
5	B	2.81	2.55	2.70	2.70
6	C	8.50	8.52	8.56	8.43
7	N	7.86	7.92	7.98	7.83
8	O	8.76	8.83	8.77	8.69
9	F	4.44	4.56	4.67	4.56
10	Ne	8.05	8.08	8.15	7.93
11	Na	6.29	6.33	6.33	6.24
12	Mg	7.54	7.58	7.58	7.60
13	Al	6.46	6.47	6.48	6.45

Figure 1. Example of the content of the **solar_abu.txt** file.

2.1. *Requested packages*

To properly work, `salvador` requires `Python` 3.9 or newer and the following packages: numpy, scipy, pandas, matplotlib, astropy, specutils, PyAstronomy, pypdf (`K2`

has its own requirements, see the K2 documentation). They are listed in the file **requirements.txt**.

You can install them from terminal via pip/pip3:

pip install -r requirements.txt

or via conda:

conda install -c conda-forge numpy scipy pandas matplotlib astropy specutils pyastronomy pypdf

The command **make check** tells you if some of them is missing.

3. USAGE: THE INPUT FILES

Here there is the explanation of how to use the code salvador. The code can work both with a grid of precomputed spectra or by calculating the synthetic on the fly with the SYNTHE code through K2. salvador is also able to run in interactive or automatic mode. In the following it will explained step by step how to use the code and how to handle with all the input files (these are present in the subdirectory **tutorial/**).

In the following will be described all the input files required by salvador and how to manipulate them in order to use the code properly in all the different modes.

All the input files, together with the spectra that you want to analyze must be present in the same directory in which you want to work.

3.1. *input.salvador*

The **input.salvador**[3] is the first input file required and is the one that sets the usage mode of salvador. In Fig. 2 is shown an example of the mandatory keywords present in the **input.salvador**. All the keywords in the left side must be separated by an "=".

The keywords must be written in capital letters, as in the figures. The values y/n of **INTER** and **CALC_BEST** can be written in small or capital letters. When it starts, salvador checks the file:

- if a mandatory keyword is missing, or a value is not valid (e.g. a text where a number is expected, or **INTER** different from y/n), salvador stops with a message that says which keyword is wrong;
- if a keyword is not recognized (for instance because it is misspelled), salvador prints a warning and suggests the most similar valid keyword (e.g. *unknown keyword CAL_BEST is ignored. Did you mean CALC_BEST?*). The keyword is ignored, so its default value is used.

[3] The input.salvador is a mandatory name for this file. This is the only mandatory name required by the code.

In Fig. 3 is shown an example of the optional keywords that can be present in the **input.salvador**. If they are missing, default values will be used.

In the following are explained the mandatory keywords.

```
OBS                 = lista.stars
NORM                = 2
WINDOW              = 5
INTER               = n
SCLIP               = 3
CALC_BEST           = n
```

Figure 2. Example of the file input.salvador with the mandatory keywords.

```
OBS                 = lista.stars
NORM                = 2
WINDOW              = 5
INTER               = n
SCLIP               = 3
CALC_BEST           = y
PATH_SYN            = /path/to/synthetic/directory/list_syn.txt
SOLAR_REF           = 3
LIMIT_ABU           = 4
EWR_MIN             = -5.5
EWR_MAX             = -4.5
EP_MIN              = 1
EP_MAX              = 6
VERBOSE             = 1
AUTO_RES            = 35
NPIX_RV             = 0.3
```

Figure 3. Example of the file input.salvador with also the optional keywords.

- **OBS**: this keyword refers to the file containing the names and properties (see Sect. 3.2 for a complete explanation) of the spectra that you want to analyze. In this example the file is named **lista.stars**, but the actual name can be whatever you want.
- **NORM**: this keyword refers to the type of normalization that you want to apply to your spectra. Here the different valid values:
  - 0: you assume that someone else (usually God) already provided you a normalized spectrum and therefore this is perfect by definition. With this value the spectra that you provide will be used as they are, without any manipulation.
  - 1: with this value each spectrum will be rigidly scaled to a synthetic spectrum used as a reference.
  - 2: with this value each spectrum will be normalized by using a linear/cubic spline[4] through the `scipy.interpolate` interp1d[5] function that pass for

specific continuum regions properly identified along the spectrum. In order to select the continuum regions to perform the normalization, in case you use NORM = 2, the code reads a specific file passed in the file containing the spectra (see Sect. 3.2 and 3.2.3 for the complete explanation).

- 3: still under development: if NORM = 3 is given, `salvador` stops at the start with a message.

- **WINDOW**: this keyword sets the size (in Å) of the region around the line under analysis in which the synthetic spectra are calculated and that will be plotted in the output .pdf file (see Sect. 4 for the explanation of the output files). If the limits of the region where the χ^2 is calculated (see Sect. 3.2.2) need more space, the region is automatically widened for that line.

- **INTER**: this keyword enables the interactive (**y**) or the automatic (**n**) mode.

- **SCLIP**: this keyword defines at how many σ the lines will be discarded at the end of the analysis when performing the calculation of the mean value.

Here are explained the optional keywords in the input.salvador file.

- **CALC_BEST**: this value allows to create 2 final synthetic spectra with a step of ±0.2 dex with respect to the final abundance value. This serve in the .pdf file to better visualize the line fit. Indeed, in the .pdf file for each line is present the best fit and 2 additional synthetic spectra less and more abundant than the best fit. If this keyword is not present (default y) the 2 final synthetic are calculated, otherwise (**CALC_BEST = n**) the less and more abundant synthetic are chosen from the already calculated synthetic spectra. In this case the code is faster because it skips the calculation of two new synthetic spectra once the convergence is reached.

- **PATH_SYN**: this keyword allows to use `salvador` with a grid of precomputed synthetic spectra. If this keyword is not present the code will calculate the synthetic spectra by calling `K2`. If a path ending with the name of a file is passed the code will use the grid of precalculated synthetic spectra to perform the analysis (see Sect. 7 for the explanation). Example of how to pass the path to the grid of synthetic:

  /path/to/the/synthetic/directory/synthetic_list

  The synthetic_list must contain a list (to the date with very strict requirements on the names) of all the synthetic spectra that will be used for the analysis. In Sect. 7 will be provided a full description of how to use `salvador` with a grid of precomputed synthetic spectra.

[4] the kind of spline depends on the number of continuum points provided. With less than 4 points a linear spline is used, otherwise a cubic one.

[5] https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.interp1d.html

- **SOLAR_REF**: this keywords allows to choose the solar reference to calculate the [Fe/H] or [X/Fe] abundance ratios. If the keyword is not present the default values from E. Caffau et al. (2011) and K. Lodders (2010) will be used (corresponding to the second column), otherwise you can use other solar references by specifying a number that indicates the column with the values present in the **solar_abu.txt** file:
  - **3**: solar reference from N. Grevesse & A. J. Sauval (1998).
  - **4**: solar reference from E. Magg et al. (2022).
  - **5**: solar reference from M. Asplund et al. (2009).
- **LIMIT_ABU**: this value allows to choose how many iterations the code performs in the automatic mode before skipping the line and passing to the next one. If this keyword is not present by default the code performs 11 iterations that correspond to a variation in the line abundance of ± 3 dex with respect to the initial value. You can choose the permitted variation in terms of $\pm$X dex with respect to the initial value before skipping the line and passing to the next one.
- **EWR_MIN**: this value represents the minimum value of reduced EW[6] (EWR) for a line. If a line has an EWR smaller than the chosen value it will be discarded from the final calculation of the abundance ratio. The default value is EWR_MIN $= -6.0$.
- **EWR_MAX**: this value represents the maximum value of EWR for a line. If a line has an EWR greater than the chosen value it will be discarded from the final calculation of the abundance ratio. The default value is EWR_MAX $= -4.4$.
- **EP_MIN**: this value represents the minimum value of excitation potential (EP) for a line. If a line has an EP smaller than the chosen value it will be discarded from the final calculation of the abundance ratio. The default value is EP_MIN $=$ 0.0.
- **EP_MAX**: this value represents the maximum value of EP for a line. If a line has an EP greater than the chosen value it will be discarded from the final calculation of the abundance ratio. The default value is EP_MAX $= 10.0$.
- **VERBOSE**: represents the level of information that is displayed on the terminal during the analysis. The default value is 0. Allowed values are 0 and 1.
- **AUTO_RES**: this keyword enables the automatic estimation of the resolution of each line to apply to the synthetic spectra, overriding any values provided in the input linelist. This feature is disabled by default, but you can enable

[6] $\log_{10}\left(\frac{\text{EW [Å]}}{\lambda \text{ [Å}}\right)$

it by providing a number **X** (strictly > 0) that allows the code to estimate the resolution within $\pm$X % from the input value. If the new resolution is within this percentage, it will be automatically changed, otherwise the original resolution is kept.

- **NPIX_RV**: this keyword enables an automatic, local radial velocity (RV) correction for the specific line under analysis. It defines a tolerance window (in pixels, including fractional values). If the centroid of the observed line deviates from the synthetic centroid but falls within this defined tolerance, the code calculates the exact offset and shifts the observed spectrum to achieve perfect alignment with the synthetic profile.
- **PLOT_TYPE**: this keyword allows to chose the style for the plot of the observed spectrum in the main plot of the PDF output. The default value is for a solid line.
  - **0**: solid line.
  - **1**: scatter plot.
  - **2**: step plot.
- **NPROC**: number of lines analyzed at the same time, on different cores of your computer. The default value is 1 (one line after the other). It is used only in automatic mode (**INTER = n**). See Sect. 9 for the explanation.
- **ROT_ABU_SPAN**, **ROT_MIN**, **ROT_MAX**, **ROT_STEP**, **ROT_NMC**: options for the measurement of the rotational velocity (keyword **ROTAT** in the linelist): range of abundance ($\pm$ dex, default 0.5; 0 to keep the abundance fixed), minimum and maximum rotational velocity (default 0 and 60 km s^{-1}), step of the grid of rotational velocities (default 10 km s^{-1}), and number of Monte Carlo simulations for the errors (default 100). See Sect. 8 for the explanation.

3.2. *lista.stars*

This file contains all the spectra and the relative information that will be used to perform the chemical analysis. In Fig. 4 is displayed the configuration of the lista.stars file without the setup for the errors arising from the atmospheric parameters, while in Fig. 5 is shown the same file but with 3 additional columns that represent the errors in the atmospheric parameters and will be used to calculate the uncertainty associated with the abundances arising from the adopted atmospheric parameters.

The layout of the lista.stars file is the following:

- First column: contains the name of the spectrum that you want to analyze. The code reads both FITS and ASCII files, but the entire name of the spectrum must be provided in this column. In the case of an ASCII file, the wavelength (in Å) and the flux must be in the first and second column[7].

```
spec_001.fits  4100  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt
spec_002.fits  4200  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt
spec_003.fits  4300  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt
spec_004.fits  4400  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt
spec_005.fits  4500  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt
spec_006.fits  4600  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt
spec_007.fits  4700  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt
spec_008.fits  4800  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt
spec_009.fits  4900  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt
spec_010.fits  5000  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt
```

Figure 4. Example of the file lista.stars without the setup for the calculation of the errors arising from the atmospheric parameters.

```
spec_001.fits  4100  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt  100  0.1  0.2
spec_002.fits  4200  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt  100  0.1  0.2
spec_003.fits  4300  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt  100  0.1  0.2
spec_004.fits  4400  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt  100  0.1  0.2
spec_005.fits  4500  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt  100  0.1  0.2
spec_006.fits  4600  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt  100  0.1  0.2
spec_007.fits  4700  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt  100  0.1  0.2
spec_008.fits  4800  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt  100  0.1  0.2
spec_009.fits  4900  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt  100  0.1  0.2
spec_010.fits  5000  0.50  -1.50  1  2.00  0.0  abu_var.txt  linelist.txt  wind_continuum.txt  100  0.1  0.2
```

Figure 5. Example of the file lista.stars with the setup for the calculation of the errors arising from the atmospheric parameters.

- Second column: is the effective temperature expressed in K.
- Third column: is the logarithm of the surface gravity.
- Fourth column: is the metallicity [M/H].
- Fifth column: is the chemical mixture that will be adopted in the calculation of the synthetic spectra. 5 different values are allowed:
  - 1: α enhancement of 0.4 dex, that means that the synthetic will have [α/Fe] = +0.4 dex.
  - 2: α enhancement of 0.2 dex, that means that the synthetic will have [α/Fe] = +0.2 dex.
  - 3: α solar-scaled, that means that the synthetic will have [α/Fe] = +0.0 dex.
  - 4: α depleted of 0.2 dex, that means that the synthetic will have [α/Fe] = −0.2 dex.
  - 5: α depleted of 0.4 dex, that means that the synthetic will have [α/Fe] = −0.4 dex.
- Sixth column: is the microturbolent velocity (ξ) expressed in km s^{-1}.

  The values from second to sixth columns will be read by `K2` to compute the proper synthetic associated to the spectrum under analysis.
- Seventh column: is the radial velocity (RV) expressed in km s^{-1} that will be applied to the spectrum.

[7] Suggestion: avoid to provide names longer than 25 characters. If you pass longer names, the code will work anyway, but the pdf titles will not look so nice.

- Eighth column: is the name of a file in which you can specify possible abundance variations of individual elements with respect to the used model atmosphere. The file lists a character keyword indicating the kind of variation, the atomic number of the element to be changed and the value of the variation according to the first keyword. In Sect. 3.2.1 will be explained in detail how to set the abundance variations and the corresponding keywords.

- Ninth column: is the name of a file that represents the linelist associated to the spectrum under analysis. In this file you pass the wavelengths and other parameters for all the lines that you want to analyze. In Sect. 3.2.2 will be explained in detail how to set the linelist file.

- Tenth column: is the name of a file representing the continuum regions present in the spectrum. This file is used only in case in which in the **input.salvador** the keyword **NORM = 2**. In all the other cases this column must be present but will be ignored by the code.

In case you want to perform also a calculation of the errors in the abundances associated to the atmospheric parameters, in the **lista.stars** file you have to include also **3 additional columns** representing the errors in **temperature, logg,** and **microturbolence**, respectively[8].

3.2.1. *Abundance variation file*

This file is present in the eighth column of the **lista.stars** file. Here you can specify variations of individual elements with respect to the model atmosphere. The file lists a character keyword indicating the kind of variation, the atomic number of the element to be changed and the value of the variation according to the first keyword. The latter can be:

- **DELTA**: for the relative abundance variation with respect to the initial abundance present in model atmosphere.

- **ABS**: for the absolute abundance A(X) in the traditional scale $A(X) = \log(N/N_H) - 12$.

- **ISOR**: for the isotopic ratio I_A/I_B of a species including only two isotopes with A<B (i.e. $^{12}C/^{13}C$);

- **ISOF**: for the isotopic fractions, in the form I_A/I_{TOT}, I_B/I_{TOT}, I_C/I_{TOT} with A<B<C.

- **GF**: to mimic the abundance variation but varying the $\log gf$ of the line.

[8] These additional columns will be read only if you pass the 3 columns. If you pass only 1 or 2 of them they are ignored and `salvador` prints a warning: the errors are NOT computed for that spectrum.

If you do not need variations of individual spectra, you can provide the name of an empty file or of a non existing file.

This file is never modified by `salvador`: the abundance variations needed during the analysis are added to a temporary copy of the file, which is passed to `K2`.

In Fig. 6 is displayed an example of the abundance variation file.

```
DELTA    13   +0.5
ABS      19   4.30
ISO      6    10
ISOF     12   0.85   0.10   0.05
GF       11   +0.3
```

Figure 6. Example of how to set the abundances variation file.

3.2.2. *Linelist file*

This file is present in the ninth column of the **lista.stars** file and contains all the lines that you want to analyze for a given spectrum. An example of the linelist file is shown in Fig. 7.

```
ABU   7461.519   26.00  30000  7461.2   7461.9  0
ABU   7489.578   22.00  30000  0        0       0
ABU   7491.646   26.00  30000  0        0       0
ABU   7496.103   22.00  30000  0        0       0
ABU   7511.018   26.00  30000  0        0       0
ABU   7555.597   28.00  30000  0        0       0
ABU   7568.897   26.00  30000  0        0       0
ABU   7574.042   28.00  30000  0        0       0
ABU   7583.788   26.00  30000  0        0       0
ABU   7586.018   26.00  30000  0        0       0
ABU   7698.966   19.00  30000  7698.45  7699.5  0.2
ABU   7710.363   26.00  30000  0        0       0
ABU2  8806.756   12.00  30000  8805.4   8806.4  0    8807.0  8807.9
```

Figure 7. Example of the linelist file.

The layout of the linelist file is the following:

- First column: indicates the kind of analysis that you want to perform. The allowed values are the following:
  - **ABU**: performs a normal analysis of an atomic line by using the core of the line (see left panel of Fig. 8).
  - **ABU2**: performs a normal analysis of an atomic line by using the wings of the line (see right panel of Fig. 8). In this case two additional columns must be present at the end of the file. When using **ABU2** you **MUST** specify the two regions corresponding to the left and right wing in which you want to perform the analysis.

  - **MOLEC**: performs a normal analysis of an molecule. The analysis is done like in **ABU**, but all the information regarding the calculation of the equivalent width (EW), the excitation potential, and log gf are skipped.
  - **ABU_GF**: performs a normal analysis of an atomic line by using the core of the line but varying the log gf of the line instead of the abundance.
  - **ABUF**: performs a normal analysis of an atomic line by using the core of the line like for the keyword **ABU**, but with the difference that it enables the use of the *eqwb* method for the final calculation of the EW (see Sect. 5 for the explanation).
  - **ISOR**: allows for the measurement of the isotopic ratios. Useful for $^{12}C/^{13}C$ or $^{6}Li/^{7}Li$.
  - **ROTAT**: measures the projected rotational velocity of the star from the line, together with the abundance of the element (see Sect. 8). Four optional columns can be added at the end of the row (see below).

- Second column: is the wavelength expressed in Å of the transition that you want to analyze.

- Third column: is the code of the transition under analysis with the following syntax: XX.00 refers to the XX element in the neutral state, XX.01 refers to the XX element in its first ionized state, etc.

- Fourth column: is the resolution at which the synthetic spectrum will be convolved in order to match the resolution of the observed spectrum.

- Fifth and sixth columns: represent the left and right limits around the line. In that region $\lambda_1 - \lambda_2$ the χ^2 will be calculated. If you pass $0 - 0$, then the limits will be automatically determined. If you pass only a left or right limit it will be ignored.

- Seventh column: is the initial abundance variation of the element under analysis that will be applied to the synthetic spectrum. This is valid for all the keywords, but **ISOR**.
  Example: if you are measuring a given abundance (Al for example) and the abundance of this element is [Al/Fe] = +1.50 dex, the first synthetic spectrum will be calculated with a solar-scaled [Al/Fe] abundance ([Al/Fe] = 0.0 dex). Some calculations of synthetic spectra are needed until the convergence is reached. Instead, with this keyword you can set the initial [Al/Fe] abundance directly to +1.50 dex by putting 1.50 in the column, thus reaching a faster convergence.
  On the other hand, if you are using the keyword **ISOR**, this value sets the initial value of the isotopic ratio that will be used to calculate the first synthetic spectrum.

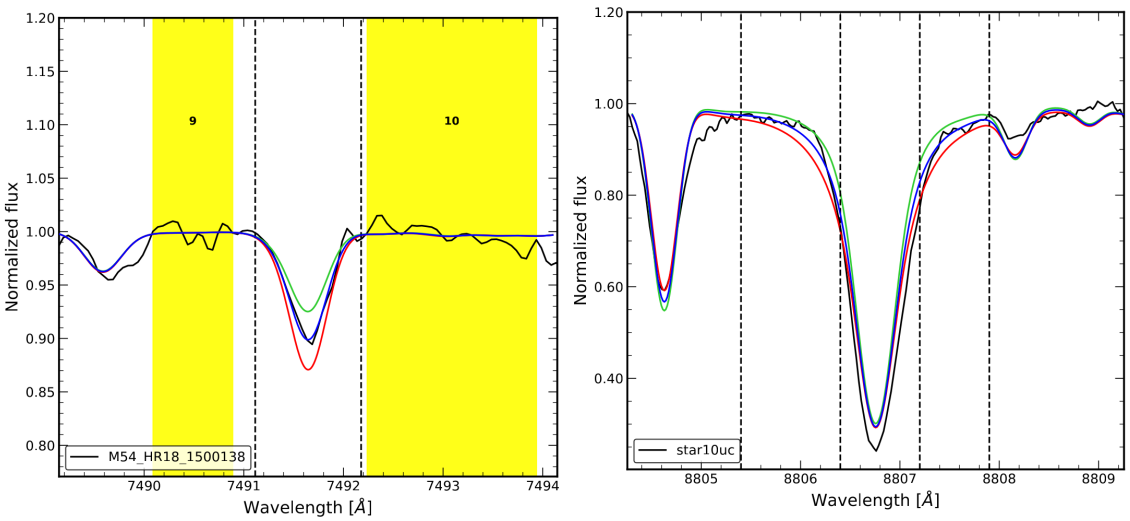


Figure 8. Example of the use of **ABU** and **ABU2** keywords.

If you are using **ABU2** in the fifth and sixth columns you must pass the limits of the left wing and in the eighth and ninth columns the limits of the right wing.

If you are using **ROTAT**, you can add four columns at the end of the row (eighth to eleventh), with the options for that line:

- Eighth column: the initial abundance A(X) of the element (absolute value).
- Ninth column: the range of abundance (± dex) around the initial value that is explored. If it is 0 the abundance is kept fixed.
- Tenth and eleventh columns: the minimum and maximum rotational velocity explored, in km s^{-1}.

For instance:

```
ROTAT  6707.875   3.00  40000  6707.150  6708.600  0  2.50  0.3  0  180
ROTAT  7511.018  26.00  40000  7510.500  7511.500  0
```

For the first line (Li) the abundance is explored between A(Li) = 2.20 and 2.80, and the rotational velocity between 0 and 180 km s^{-1}. For the second line (Fe) these columns are not given, so the initial abundance is that of the model atmosphere (plus the seventh column) and the other options are taken from the **input.salvador** file (keywords **ROT_ABU_SPAN**, **ROT_MIN**, **ROT_MAX**) or their default values. For the **ROTAT** lines the limits of the fifth and sixth columns must include the whole line broadened by the rotation: the limits determined automatically $(0-0)$ are based only on the instrumental resolution, and are too narrow for a rotating star.

`salvador` checks the number of columns of each row of the linelist (7; 9 for **ABU2**; 11 for **ROTAT** with its options) and prints a warning for the rows with a different number of columns.

3.2.3. *Continuum regions*

The continuum regions file is present in the tenth column of the **lista.stars** file and consists of N rows and 2 columns for row (see Fig. 9), in which the first and second columns represent the λ_1 and λ_2 of the continuum region interval, expressed in Å.

```
7456.62          7457.14
7458.38          7460.75
7463.05          7468.59
7470.42          7470.75
7475.41          7478.38
7480.05          7481.05
7484.60          7488.90
7490.07          7490.90
7492.22          7493.95
7496.77          7497.97
7499.52          7500.63
7503.17          7504.26
7507.96          7510.26
7512.72          7513.75
7514.69          7517.48
7519.14          7522.02
7523.42          7524.47
7525.85          7528.83
7529.43          7530.48
7531.88          7535.57
7536.41          7538.95
7541.08          7543.83
7544.51          7545.68
7548.32          7551.15
```

Figure 9. Example of the continuum regions file.

You can create the continuum regions file in any way you prefer, but I created a simple `Python` script named **cows.py** that allows to select the continuum regions from an observed spectrum in an interactive mode and saves the regions in a file, as requested by `salvador`. See the Appendix A for an explanation of how **cows.py** works. **cows.py** is one of the files that you download.

3.3. *Linelist for `K2`*

The file **linelist.xkur** must to be present in the working directory (at the beginning is present into the subdirectory **tutorial**). This file is required by `K2` to calculate in a correct way the synthetic spectra. Indeed, it includes the complete path of the linelist files. If this file does not exist in the working directory (or if it is empty) the synthetic spectrum will be calculated but without including atomic and molecular transitions (only continuum). The lists of the files for the atomic and molecular lines must be preceded by the line #ATOMS and #MOLECULES, respectively. TiO and H_2O molecules can be enabled with the lines #TIO and #H2O, respectively in

the same file. These molecules, due to the huge number of transitions, should be included only for specific cases in order to avoid wast of time and space: TiO is relevant for T_{eff} below 4000 K, while H_2O is relevant for cool stars with high logg (for instance M dwarf stars). Finally, predicted transitions can be included with the line #PREDICTED. This option should be used only to calculate low resolution spectra (energy distributions, useful to derive integrated colors). Fig. 10 shows some examples of this input file.

```
#ATOMS
/home/deimer/softw/kurucz/atoms/atoms_0_100.dat
/home/deimer/softw/kurucz/atoms/atoms_100_200.dat
/home/deimer/softw/kurucz/atoms/atoms_200_300.dat
/home/deimer/softw/kurucz/atoms/atoms_300_400.dat
/home/deimer/softw/kurucz/atoms/atoms_400_500.dat
/home/deimer/softw/kurucz/atoms/atoms_500_600.dat
/home/deimer/softw/kurucz/atoms/atoms_600_700.dat
/home/deimer/softw/kurucz/atoms/atoms_700_800.dat
/home/deimer/softw/kurucz/atoms/atoms_800_900.dat
/home/deimer/softw/kurucz/atoms/atoms_900_1000.dat
/home/deimer/softw/kurucz/atoms/atoms_1000_3000.dat
/home/deimer/softw/kurucz/atoms/atoms_3000_end.dat
#MOLECULES
/home/deimer/softw/kurucz/molecules/c2ax.am
/home/deimer/softw/kurucz/molecules/c2ba.am
/home/deimer/softw/kurucz/molecules/c2dabrookek.am
/home/deimer/softw/kurucz/molecules/c2ea.am
/home/deimer/softw/kurucz/molecules/chmasseron2.am
/home/deimer/softw/kurucz/molecules/cnaxbrookek.am
/home/deimer/softw/kurucz/molecules/cnbxbrookek.am
/home/deimer/softw/kurucz/molecules/cnxx12brooke.am
/home/deimer/softw/kurucz/molecules/coax.am
/home/deimer/softw/kurucz/molecules/coxx.am
/home/deimer/softw/kurucz/molecules/mgh.am
/home/deimer/softw/kurucz/molecules/nh_ber_kur.am
/home/deimer/softw/kurucz/molecules/ohupdate.am
```

Figure 10. Example of the input file listing the linelists used in the calculation of synthetic spectra.

Generally, for most of the purposes (and in particular for the synthetic spectra used in the chemical abundance analyses) only #ATOMS and #MOLECULES should be used.

N.B. you have to specify your path where the linelists are located.

To make the calculation faster, for each line `salvador` passes to `K2` a reduced copy of the linelists, which contains only the transitions within 50 Å from the region of the line. The synthetic spectra are identical to those calculated with the complete linelists, but `K2` has to read only a small part of the files. These copies are written in the directory **linelist_cut** of the working directory, which is removed at the end of the run. The first time a linelist file is used, `salvador` checks it (only once, the result is saved in the file ∼**/.cache/salvador/linelist_index.json**). The file **linelist.xkur** itself is never modified.

Once that you ran `salvador`, either in the interactive or automatic mode, the following files and directory will be produced:

- synth_out: is a directory in which all the unbroadened synthetic calculated by `K2` are stored, together with the best fit of each line (see Sect. 5).
- For each spectrum a ROOT_NAME.fit file is generated, that contains the information about the lines analyzed, their abundances and other parameters. In Sect. 4.1 will be explained in detail the content of the .fit file. If for some lines you measured the isotopic ratio also a second file named ROOT_NAME_isor.fit is created. If for some lines you measured the rotational velocity (**ROTAT**), also a file named ROOT_NAME_rot.fit is created (see Sect. 4.2).
- For each spectrum a ROOT_NAME.pdf file is generated. This file contains the plots of all the lines that were analyzed. In Fig. 11 is displayed an example of the .pdf output for one line of Fe. N.B. in the plots, the yellow regions are displayed only when the keyword NORM = 2, in all the other cases they are not present.
- abundance_final.txt: this is a file containing, for all the analyzed spectra, the abundances of the analyzed elements in the form [Fe/H] or [X/Fe], together with the corresponding errors. In Sect. 4.3 will be explained in detail the content of the abundance_final.txt file. If for some lines you measured the isotopic ratio also a second file named abundance_final_isor.txt is created.

During the run also the following directories are used: **tmp_plots** (the plots of the single lines, before they are merged in the .pdf file) and **linelist_cut** (see Sect. 3.3), which are removed at the end of the run, and, only with **NPROC** > 1, **salvador_work**, which contains a log file for each process with the detailed output of its lines (see Sect. 9) and is kept until the following run.

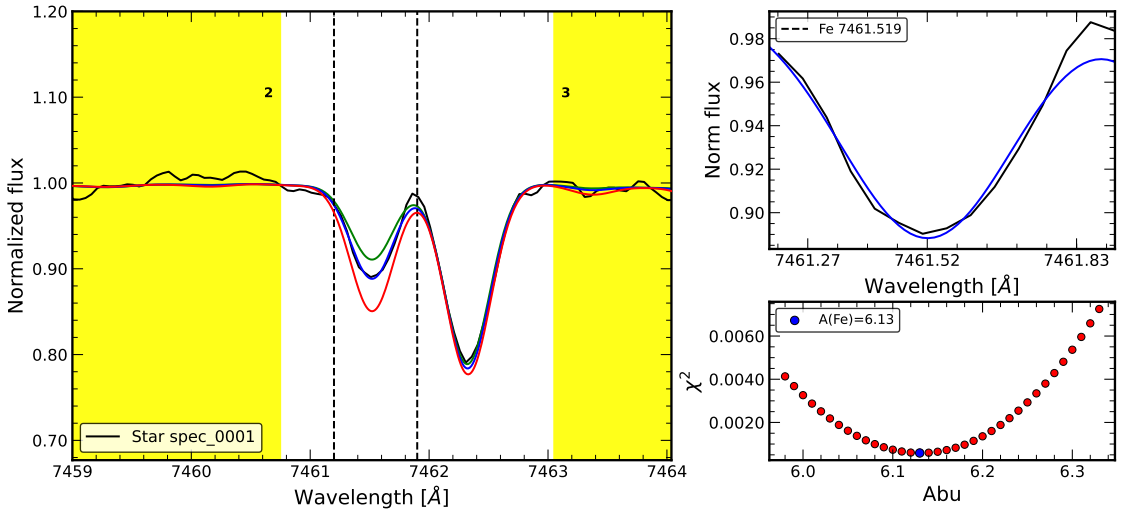


Figure 11. Example of the fit of a line present in the .pdf file.

4.1. *.fit file*

The ROOT_NAME.fit and the ROOT_NAME_isor.fit (if present) file(s) contains, for each spectrum, the following information:

- First column: the root name of the spectrum.
- Second column: the wavelength expressed in Å of the line analyzed.
- Third column: the code of the line analyzed.
- Fourth column: the absolute A(X) abundance or the isotopic ratio.
- Fifth column: the EW of the line in mÅ.
- Sixth column: the EP of the line.
- Seventh column: the log gf of the line.
- Eighth column: the error arising from the Monte Carlo simulation. See Sect. 5 for a detailed discussion.

Columns five, six, and seven are not present in the file ROOT_NAME_isor.fit.

Only in case the errors associated to the atmospheric parameters are provided in the **lista.stars** file 3 additional columns will be present in the .fit files:

- Ninth (sixth) column: the error in the abundance associated to a variation of $+\Delta$T. In this case the abundance is determined again, varying only T, while keeping logg and ξ fixed.
- Tenth (seventh) column: the error in the abundance associated to a variation of $+\Delta$ logg. In this case the abundance is determined again, varying only logg, while keeping T and ξ fixed.
- Eleventh (eighth) column: the error in the abundance associated to a variation of $+\Delta\xi$. In this case the abundance is determined again, varying only ξ, while keeping T and logg fixed.

Moreover, as a final column (ninth (sixth) or twelfth (ninth) depending on the presence of the errors in the atmospheric parameters) is provided a flag value for the lines. The possible values are:

- **-3**: the line is discarded because its abundance was not calculated (ABU = 99.99).
- **-2**: the line is discarded because it is outside the limits in EP (see Sect. 3.1).
- **-1**: the line is discarded because it is outside the limits in EWR (see Sect. 3.1).
- **0**: the line is discarded because it is a more than Nσ from the mean value (see keyword **SCLIP**).

- **1**: the line is used for the calculation of the final abundance ratio.

flags −2 and −1 are not present in the ROOT_NAME_isor.fit file.

A detailed explanation of how the abundance errors due to the atmospheric parameters are computed will be provided in Sect. 5.

Examples of the .fit file are provided in Figs. 12 and 13

#SPECTRUM	LINE	ELEM	ABU	EW	EP	log_gf	eMC	flag_line
spec_001	7385.245	28.00	4.31	41.74	2.740	-2.070	99.99	1
spec_001	7389.398	26.00	5.55	78.11	4.301	-0.458	99.99	1
spec_001	7393.610	28.00	4.83	83.12	3.606	-0.758	99.99	0
spec_001	7400.180	24.00	3.79	86.33	2.899	-0.111	0.08	1
spec_001	7401.683	26.00	5.27	17.94	4.186	-1.353	99.99	1
spec_001	7405.772	14.00	6.13	35.95	5.613	-0.620	0.03	1
spec_001	7411.153	26.00	5.46	75.94	4.283	-0.430	99.99	1
spec_001	7414.518	28.00	4.29	100.04	1.986	-2.480	99.99	1
spec_001	7418.666	26.00	5.42	25.64	4.143	-1.376	99.99	1
spec_001	7422.281	28.00	4.50	97.10	3.635	-0.155	99.99	1
spec_001	7423.496	14.00	6.35	50.37	5.619	-0.530	0.02	1
spec_001	7445.749	26.00	5.44	91.92	4.256	-0.230	99.99	1
spec_001	7461.519	26.00	5.59	40.29	2.559	-3.585	99.99	1
spec_001	7491.647	26.00	5.71	47.17	4.301	-1.060	99.99	1

Figure 12. Example of the output present in the .fit file when the errors associated with the atmospheric parameters are not provided.

#SPECTRUM	LINE	ELEM	ABU	EW	EP	log_gf	eMC	d_Teff	d_logg	d_vt	flag_line
spec_001	7385.245	28.00	4.31	41.74	2.740	-2.070	99.99	0.08	0.00	0.01	1
spec_001	7389.398	26.00	5.55	78.11	4.301	-0.458	99.99	0.07	0.00	0.01	1
spec_001	7393.610	28.00	4.83	83.12	3.606	-0.758	99.99	0.06	0.00	0.02	1
spec_001	7400.180	24.00	3.79	86.33	2.899	-0.111	0.08	0.08	0.00	0.01	1
spec_001	7401.683	26.00	5.27	17.94	4.186	-1.353	99.99	0.09	0.00	0.01	1
spec_001	7405.772	14.00	6.13	35.95	5.613	-0.620	0.03	0.05	0.00	0.01	1
spec_001	7411.153	26.00	5.46	75.94	4.283	-0.430	99.99	0.06	0.00	0.00	0
spec_001	7414.518	28.00	4.29	100.04	1.986	-2.480	99.99	0.08	0.00	0.01	1
spec_001	7418.666	26.00	5.42	25.64	4.143	-1.376	99.99	0.07	0.00	0.02	1
spec_001	7422.281	28.00	4.50	97.10	3.635	-0.155	99.99	0.08	0.00	0.01	1
spec_001	7423.496	14.00	6.35	50.37	5.619	-0.530	0.02	0.08	0.00	0.01	1
spec_001	7445.749	26.00	5.44	91.92	4.256	-0.230	99.99	0.10	0.00	0.01	1
spec_001	7461.519	26.00	5.59	40.29	2.559	-3.585	99.99	0.08	0.00	0.02	1
spec_001	7491.647	26.00	5.71	47.17	4.301	-1.060	99.99	0.04	0.00	0.02	1

Figure 13. Example of the output present in the .fit file when the errors associated with the atmospheric parameters are provided.

In Appendix B is provided the description of a `Python` script that allows the user to merge together different **.fit** files belonging to the same star.

4.2. *_rot.fit file*

The ROOT_NAME_rot.fit file is created only if some lines of the linelist have the keyword **ROTAT** (see Sect. 8). It contains, for each of these lines:

- First column: the root name of the spectrum.
- Second column: the wavelength expressed in Å of the line analyzed.
- Third column: the code of the line analyzed.
- Fourth column (ROT): the projected rotational velocity, in km s^{-1}.
- Fifth column (eROT): its error from the Monte Carlo simulation, in km s^{-1}.
- Sixth column (ABU): the absolute abundance A(X) found together with the rotational velocity.

- Seventh column (eABU): its error from the Monte Carlo simulation (0.00 if the abundance is kept fixed).

A line rejected in interactive mode (keyword **r**) has 99.99 in all four columns. The **ROTAT** lines are not written in the ROOT_NAME.fit file and are not used for the abundances in the **abundance_final.txt** file.

4.3. *abundance_final.txt*

This file contains the final abundances and the corresponding errors for all the sample stars and the corresponding elements included in the **lista.stars** and in the linelist file. The file is structured in the following way:

- First column: the root name of the spectrum.
- Second column: the code of the element.
- Third column: the number of lines that were used to determine the final abundance.
- Fourth column: the final abundance in the [Fe/H] or [X/Fe] mode.
- Fifth column: the final error associated to the abundance.
- Sixth column: the internal error, that is calculated in the following way:
  - If only 1 line for a given element is present, the error is the error arising from the Monte Carlo simulation. In particular a total of 200 simulations are performed.
  - If more than 1 line for a given element is present, then the error is calculated as:

$$\sigma_{\rm int} = \sqrt{\frac{\sigma_{\rm X}^2}{\rm N_X} + \frac{\sigma_{\rm Fe}^2}{\rm N_{Fe}}} \tag{1}$$

    where $\sigma_{\rm X}$ is the standard deviation of the the mean abundance of the element X, $\rm N_X$ the number of lines used to compute the abundance of X, $\sigma_{\rm Fe}$ the standard deviation of Fe, and $\rm N_{Fe}$ the number of lines used to determine the abundance of Fe.

- Seventh column: the error associated with the atmospheric parameters. This error is calculated as follows (when errors in atmospheric parameters are provided):

$$\sigma_{\rm [Fe/H]} = \sqrt{(\delta_{\rm Fe}^{\rm T_{eff}})^2 + (\delta_{\rm Fe}^{\rm log\,g})^2 + (\delta_{\rm Fe}^{\xi})^2} \tag{2}$$

$$\sigma_{\rm [X/Fe]} = \sqrt{(\delta_{\rm X}^{\rm T_{eff}} - \delta_{\rm Fe}^{\rm T_{eff}})^2 + (\delta_{\rm X}^{\rm log\,g} - \delta_{\rm Fe}^{\rm log\,g})^2 + (\delta_{\rm X}^{\xi} - \delta_{\rm Fe}^{\xi})^2} \tag{3}$$

- Eighth column: the absolute abundance in the form A(X).

- Ninth column: the final error relative to the abundance in the form [X/H]. It is calculated as the sum in quadrature of the following two errors (internal and relative to the parameters).
- Tenth column: the internal error, relative to the abundance [X/H]. It is calculated as for the sixth column, but avoiding the part relative to Fe.
- Eleventh column: the error associated with the atmospheric parameters for the abundance [X/H]. It is calculated as for the seventh column but avoiding the part relative to Fe.

The final error present in the fifth column is calculated as the sum in quadrature of the internal error and the error due to the atmospheric parameters, as follows:

$$\sigma_{[\mathrm{Fe/H}]} = \sqrt{\frac{\sigma_{\mathrm{Fe}}^2}{\mathrm{N_{Fe}}} + (\delta_{\mathrm{Fe}}^{\mathrm{T_{eff}}})^2 + (\delta_{\mathrm{Fe}}^{\log g})^2 + (\delta_{\mathrm{Fe}}^{\xi})^2} \tag{4}$$

$$\sigma_{[\mathrm{X/Fe}]} = \sqrt{\frac{\sigma_{\mathrm{X}}^2}{\mathrm{N_X}} + \frac{\sigma_{\mathrm{Fe}}^2}{\mathrm{N_{Fe}}} + (\delta_{\mathrm{X}}^{\mathrm{T_{eff}}} - \delta_{\mathrm{Fe}}^{\mathrm{T_{eff}}})^2 + (\delta_{\mathrm{X}}^{\log g} - \delta_{\mathrm{Fe}}^{\log g})^2 + (\delta_{\mathrm{X}}^{\xi} - \delta_{\mathrm{Fe}}^{\xi})^2} \tag{5}$$

An example of the **abundance_final.txt** is shown in Fig. 14.

If you are measuring also the isotopic ratio for some lines, a second file named **abundance_final_isor.txt** is created. It has the same structure of **abundance_final.txt**, with the exceptions that instead of the abundance is reported the isotopic ratio and that the last four columns are not present.

```
#STAR                          CODE  N_LIN    ABU  ERROR  E_INT  E_PARAM
spec_001          26.00      58  -1.95   0.02   0.02  99.99
spec_001          11.00       2   0.35   0.04   0.04  99.99
spec_001          12.00       3   0.70   0.15   0.15  99.99
spec_001          13.00       2  -0.51   0.02   0.02  99.99
spec_001          14.00       7   0.52   0.06   0.06  99.99
spec_001          19.00       1   0.40   0.04   0.04  99.99
spec_001          20.01       1   1.13   0.04   0.04  99.99
spec_001          22.00      11   0.35   0.04   0.04  99.99
spec_001          24.00       1   0.09   0.08   0.08  99.99
spec_001          26.01       1  -1.37   0.12   0.12  99.99
spec_001          27.00       1  -0.03   0.02   0.02  99.99
spec_001          28.00      12   0.08   0.03   0.03  99.99
########################################
```

Figure 14. Example of the output of the abundance_final.txt file, without the errors in the atmospheric parameters.

5. WORKING FLOW

Here there is the explanation of how the code works in all the different setups (interactive/automatic mode, precomputed grid of synthetic or synthetic calculated on the fly by calling `K2`).

As a first step all the information provided in the input files are read, including the spectrum, the associated parameters, and the lines to be analyzed. Then, if

in the linelist file there are lines outside the extension of the spectrum these are automatically excluded. For the valid lines, the code prints the wavelength (in Å) and the corresponding element.

According to the normalization type the spectrum is normalized (**NORM = 1/2**) or not (**NORM = 0**), and eventually radially velocity corrected. At this point, for each line `salvador` calls `K2` which through the code `SYNTHE` calculates synthetic spectra using an ATLAS9 model atmosphere. The main ingredients to calculate a synthetic spectrum are the stellar parameters (T_{eff}, logg, ξ, [M/H], and chemical mixture α) defining the model atmosphere and the linelists (including all the atomic and molecular transitions to be included in the calculation and their main parameters). The first synthetic spectrum is calculated with the chemical abundance defined in the input linelist file and according to the the chosen chemical mixture. The synthetic spectrum is calculated at very high resolution (600,000) and then convolved with a Gaussian function at the resolution provided in the input file. Then it is resampled at the same pixel step of the observed. At this point a Gaussian fit of the line is performed both on the normalized (and wavelength shifted) and synthetic spectrum and the corresponding area is calculated by direct integration[9]. A simple comparison is done:

- If the area of the line on the observed spectrum is higher than the area of the same line on the synthetic that means that the abundance of the synthetic must be increased (see the green area of Fig. 15). In this case a second synthetic spectrum is calculated by varying the abundance of the element by +0.40 dex and the procedure of comparison is repeated (see red and blue areas of Fig. 15).
  - If the area of the line on the observed spectrum is smaller than that on the synthetic (see the red area of Fig. 15), a third synthetic is calculated by varying the abundance of the element by +0.20 dex with respect to the initial value. With the 3 synthetic available a fine grid of synthetic spectra is determined by interpolating in step of 0.01 dex and the final abundance is determined by performing the χ^2 minimization between the observed spectrum and the grid (see top part of Fig. 15 where the convergence is reached).
  - On the other hand, if the area of the line on the observed spectrum is still higher than that on the synthetic (see the blue area of Fig. 15), a new synthetic is calculated by varying the abundance of +0.80 with respect to the initial value (see the yellow and purple areas of Fig. 15). At this point automatically another synthetic is calculated with an abundance variation of +0.60 dex. With the 3 synthetic available (Δabu = +0.40, +0.60, +0.80 dex) the fine grid is calculated and the procedure of χ^2 minimization is performed. If the minimum is not found, then the synthetic with the lowest

[9] https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.trapezoid.html

abundance (Δabu = +0.40 dex) is discarded and a new one with an increased abundance of +1.0 dex with respect to the initial value is calculated and the procedure of grid calculation and χ^2 minimization is repeated. This step goes on until the convergence is reached or until the abundance is varied by +3.0 dex with respect to the initial value. In this latter case the calculation is stopped, the line is discarded and the code moves to the following line.

- If the area of the line on the observed spectrum is smaller than the area of the same line on the synthetic that means that the abundance of the synthetic must be decreased. The procedure is repeated as described above, but with a decreasing abundance values.

The idea to discard a line that differs by $\pm$3.0 dex with respect to the initial value is thought in order to avoid the code to enter into a infinite loop. The fine grid of synthetic is created by interpolating among 3 synthetic spectra separated by 0.20 dex. This value is chosen in order to allow a correct interpolation of the flux. Indeed, use synthetic spectra separated by more than 0.20 dex to interpolate could lead to incorrect values of flux in the interpolated synthetic spectra. Since the grid of synthetic spectra is created starting from 3 synthetic separated by 0.20 dex in abundance, the trick of the comparison between the area of the line on the observed and the synthetic spectra allows us to make the code a bit faster without moving blinding in steps of 0.20 dex in the calculation of synthetic spectra.

Once the final abundance A(X) is determined, the EW of the line is measured. A synthetic spectrum is calculated using the same parameters used for the abundance but including only the specific transition of the line under analysis and the corresponding abundance A(X). The EW can be calculated in two different ways: using the *eqw* or the *eqwb* methods. The keyword *eqw* includes in the calculation only the transition specified by the user. In case of a transition characterized by isotopic/hyperfine splitting, all the sub-transitions related to the same transition are recognized because they have the same log gf and χ. When the EW is calculated through the *eqwb* method, all the transitions of the same species, even if they have different atomic parameters are included. This option is specific for some features that are blending of different transitions of the same species (for instance the Li resonance doublet at 6708 $\mathring{A}$ Once the synthetic is calculated it is convolved with a Gaussian function at the resolution provided in the input file. Finally, the EW is calculated by integration of the synthetic profile. Also the EP and loggf of the transition are provided by searching into the linelists provided into the **linelist.xkur** file.

Once that also the EW is determined, one of the following things can happen:

- If for the element under analysis there are more than 10 lines in the linelist, `salvador` moves to the following line. In this case, at the end of the analysis the internal error is estimated as $\sigma/\sqrt{\mathrm{N_{lin}}}$, where σ is the standard deviation

of the the mean abundance and N_{lin} the number of lines used to determine the abundance, after a σ-clipping is performed.

- If for the element under analysis there are less than 10 lines in the linelist, then a Monte Carlo simulation is performed. The best fit of the analysis is taken and noise is injected, according to the signal-to-noise ratio (SNR) of the observed spectrum around the line. At this point the procedure of abundance determination is repeated, using as observed spectrum the noise injected synthetic. The procedure is repeated a total of 200 times. The noise is generated starting from a number (seed) obtained from the name of the spectrum, the wavelength, and the element of the line: therefore the same input gives always the same error, also if the analysis is repeated or the lines are analyzed in a different order. The choice to perform a Monte Carlo simulation when less than 10 lines are present for a given element is given by the fact that in the final abundance determination a σ-clipping is performed and every feature deviating more than $N \times \sigma$ from the average is rejected. We estimate that when 10 or less lines are present for a given element, at least one line survives. If more than one line survives to the σ-clipping, the internal error is calculated as above, otherwise if only one line remains the standard deviation of the mean value is taken as the internal error in the final abundance.

If the Monte Carlo simulation is performed the error of the calculation is taken and saved in the **ROOT_NAME**.fit file under the keyword eMC, otherwise the value is automatically set to 99.99.

If in the input files also the errors in the atmospheric parameters are provided, then after the abundance A(X) is determined the procedure of analysis is repeated by varying the atmospheric parameters by the corresponding error. In this way also the errors arising by the errors in the adopted atmospheric parameters are taken into account.

- In the first run the logg and ξ are kept fixed to their best values and only the T_{eff} is varied by its uncertainty. With these new set of parameters the procedure of abundance determination is repeated. The difference between this abundance and the original one is taken as the error in the abundance due to the temperature.

- In the second run, if the element under analysis in the ionized state T_{eff} and ξ are kept fixed to their best values and only the logg is varied. Again, the analysis is repeated and the difference in abundance is taken as the error due to the logg. On the other hand, if the element is in the neutral state this step is ignored and the error in abundance due to logg is set to 0.0 dex. This is due to two main factors, one physical and the other one computational. The physical reason is linked to the fact that a change in logg has a direct impact in the electronic pressure (P_e), which in turn has an impact in the ionization

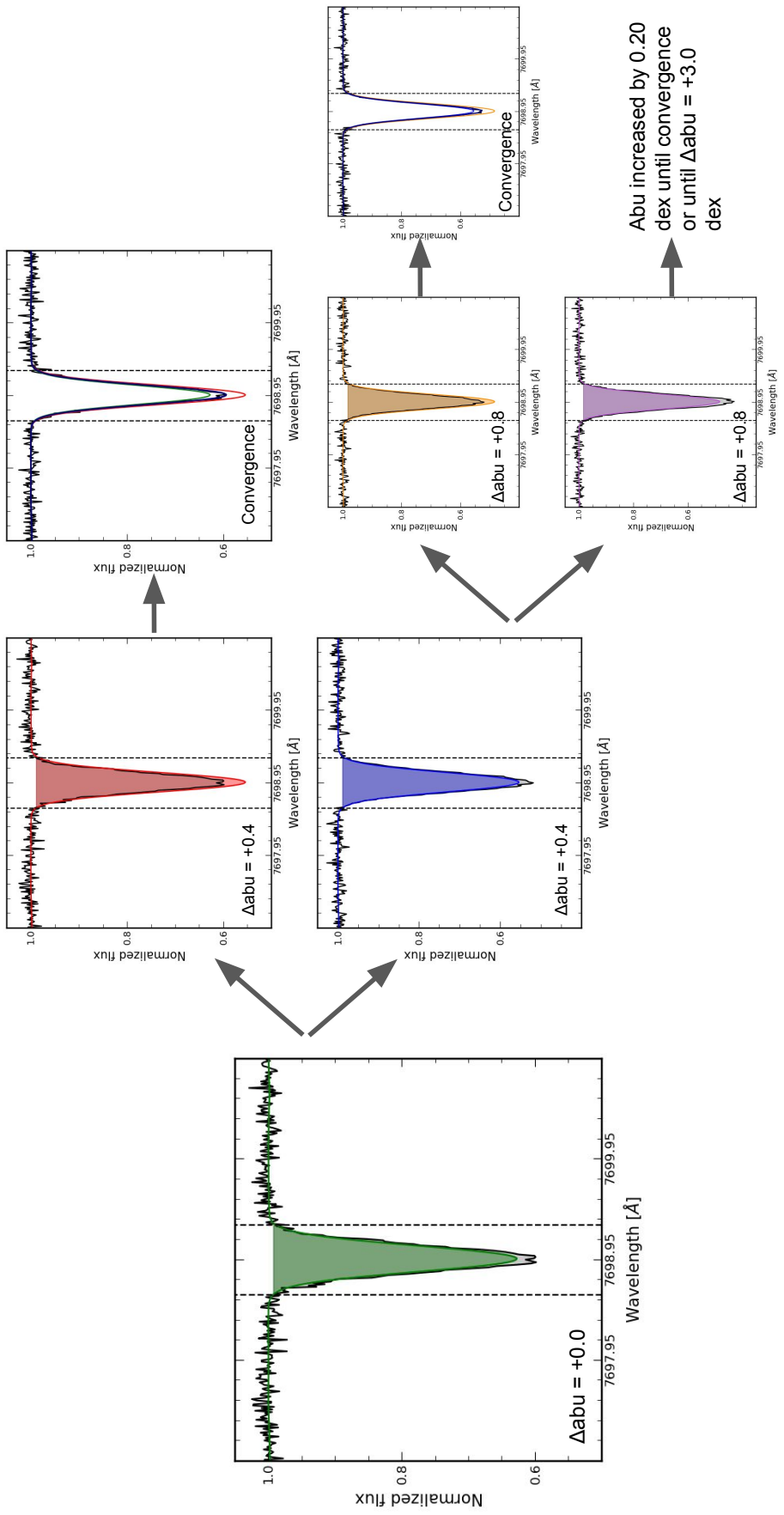


Figure 15. Working flow of `salvador`.

equilibrium. Therefore, ionized elements are more sensitive to small changes in logg than neutral counterparts. The computational reason instead is linked to the fact that the calculation of synthetic spectra to determine abundances is time consuming. With this little trick in case of a neutral element a certain amount of time is saved.

- In the last run, if the line has an EW larger than a certain value (in this case our choice is 50 mÅ), T_{eff} and logg are kept fixed to their best values and only the ξ is varied. Again, the analysis is repeated and the difference in abundance is taken as the error due to the ξ. If the line has a EW < 50 mÅ the error in the abundance due to ξ is set to 0.0 dex. As for the logg variation this is a trick due to a physical and computational reasons. In this case the physical reason is due to the fact that only lines starting to deviate from the linear part of the curve of growth are sensitive to variations in ξ. Such lines are those with a high EW. The choice of 50 mÅ is a good compromise.

The errors arising from the variations in the atmospheric parameters are stored in the **ROOT_NAME**.fit file under the keywords d_Teff, d_logg, and d_vt. In these additional runs only the abundance is determined: the plot, the EW, the Monte Carlo simulation, and the synthetic spectra at ± 0.2 dex from the best fit (keyword **CALC_BEST**) are needed only for the nominal parameters and are skipped. The model atmospheres with the varied parameters are calculated by `K2` the first time they are needed and then reused.

This procedure of analysis is repeated for every line associated to a given spectrum. After the entire analysis finishes the final abundances (in the form [Fe/H] or [X/Fe]) and the associated errors are calculated for each spectrum and stored in the **abundance_final.txt** file. For each element the final abundance is determined through a σ-clipping, with the values deviating more than $N \times \sigma$ from the average that are rejected. The error is calculated according to all the caveats described above.

- If errors in the atmospheric parameters are not provided, only the internal error is used. This is calculated as $\sigma/\sqrt{N_{lin}}$ if more than one line is present or as the error in the Monte Carlo simulation if only one line is present.

- If errors in the atmospheric are provided, the final error is calculated according to Eq. 4 or 5 depending on the element (Fe or X), with the caveat that the internal error is calculated as described in the point above.

Also the final number of lines used are saved in the **abundance_final.txt** file.

For each spectrum also a **ROOT_NAME**.pdf is created, where for each line the plot of the χ^2 minimization is saved (see Fig. 11 as an example for one line).

If in the analysis were measured Fe lines (Fe I or Fe II), at the end also the following plots are calculated and stored in the **ROOT_NAME**.pdf (see Fig. 16):

- A(Fe) vs wavelength

- A(Fe) vs EP. This plot gives an indication of the reliability of the used T_{eff}.
- A(Fe) vs reduced EW. This plot gives an indication of the reliability of the used ξ.

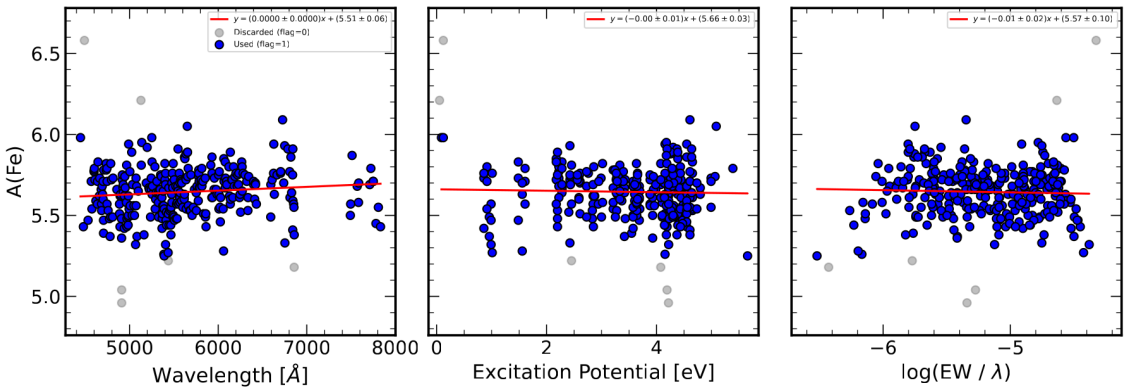


Figure 16. Example final plot that is generated when Fe lines are measured. Blue circles are for the Fe I lines, while red triangles for the Fe II. Gray points are the discarded lines.

All the synthetic (and NOT broadened) spectra created during the analysis are stored in the directory **synth_out**. If the analysis is repeated, `salvador` checks if for a given spectrum and a given line the corresponding synthetic spectra are already present in the **synth_out** directory, in order to avoid the creation of new synthetic spectra. The name of each synthetic spectrum contains all the parameters that define it (atmospheric parameters, chemical mixture, spectral range, and abundance variations), so a synthetic spectrum is reused only when it is exactly the same. The only operation that is done on the synthetic is the convolution with the Gaussian function.

Additionally, in the **synth_out** directory, for each line, is stored a file named **bestfit_ROOT_NAME_line_LINE.txt** which contains the convolved bestfit synthetic spectrum for a given line (always the one obtained with the nominal atmospheric parameters; not with a grid of precomputed synthetic spectra). The file is organized as follows: the first column contains the code of the element in the Kurucz format, the second column the wavelength of the line, the third the common wavelength for both the observed and the synthetic spectrum, the fourth the convolved bestfit flux of the synthetic, and the fifth the flux of the observed spectrum.

5.1. *Speed of the calculation*

Most of the time of the analysis is spent by `K2` and `SYNTHE` to calculate the synthetic spectra. To reduce it, `salvador`:

- runs `K2` inside its own `Python` process, instead of starting a new one (and loading again all the `Python` packages) for each synthetic spectrum;
- passes to `K2`, for each line, reduced copies of the linelists (see Sect. 3.3);

- checks first if the model atmosphere already exists, and in that case skips the interpolation of the opacity distribution functions that `K2` would perform before this check (for metallicities that are not in the grid of the model atmospheres);
- reuses the synthetic spectra already present in **synth_out**;
- in automatic mode, can analyze several lines at the same time (keyword **NPROC**, see Sect. 9).

None of these changes the results. The first three can be switched off for checks or troubleshooting, by setting to False the variables **K2_IN_PROCESS**, **TRIM_LINELIST**, and **K2_FAST_MODEL_CHECK** at the beginning of the file **salvador_lib/k2_interface.py**. The script **check_k2_speedup.py** verifies that the synthetic spectra and the EWs are the same with and without them (see Sect. 10).

6. INTERACTIVE MODE

If you are running `salvador` in interactive mode, by calling `K2`, in your working directory (once that you prepared all the input files in the proper way) open a terminal and run the command salvador.py. In Fig. 17 there is an example of how the terminal looks like at the beginning. The code prints information about the name of the spectrum under analysis, T_{eff}, logg, ξ, and its extension.

```
(base) deimer@deimer:~/softw/SALVADOR/tutorial$ SALVADOR.py

########################################
OBSERVED SPECTRUM =  spec_0001.fits
TEFF  = 4248    [K]
LOGG  = 0.77    [dex]
VT    = 1.95    [km/s]
START = 7456.16 [Å]
END   = 7879.23 [Å]
----------------------------------------
--- Lines excluded for being outside the spectral range ---
Line 6355.028 outside the spectrum.
Line 6369.459 outside the spectrum.
Line 6380.743 outside the spectrum.
Line 6390.455 outside the spectrum.
Line 6393.600 outside the spectrum.

----------------------------------------
LINE = 7461.519 [Å] ---  ELEM:  Fe
```

Figure 17. Example of `salvador` at the beginning.

Once the spectrum is normalized and the 3 synthetic spectra are calculated, `salvador` displays the plot as in Fig. 11 and in the terminal many options are displayed (see Fig. 18), allowing to intervene immediately on the observed spectrum. These are all the options available:

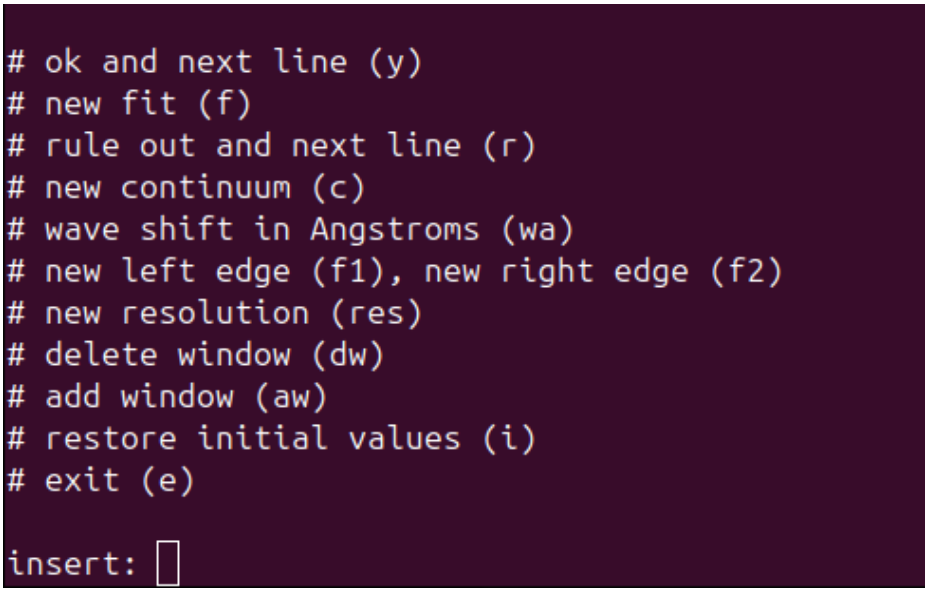


Figure 18. Options to intervene on the observed spectrum.

- **y**: this keyword allows to save all the information for a given line (abundance, EW, plot, etc.) and move to the next line, once you are satisfied of the fit.

- **r**: this keyword allows to discard a bad line (saturated, too weak, very low SNR, etc) and move to the next line.

- **f**: this keyword calculates a new synthetic spectrum, in order to move the 3 synthetic spectra of the fit by one step (0.20 dex, or 10 for **ISOR**, where the ratio cannot go below 1) towards the side where the χ^2 decreases, as done automatically in the automatic mode. If the minimum of the χ^2 is already inside the grid no new synthetic spectrum is needed and a message is printed. This keyword is not available with a grid of precomputed synthetic spectra and for the **ROTAT** lines.

- **c**: this keyword allows to multiply the normalized spectrum by a certain factor in order to rigidly move it. You can use it if you are not satisfied by the normalization done by the code.

- **wa**: this keyword allows to shift (in Å) the observed spectrum. You can use to correct small shifts in wavelength between the observed and synthetic spectra.

- **f1, f2**: these keywords allow to modify the left and right extensions of the region where the line is defined. This region is the one used to calculate the χ^2. If **ABU2** is used also **f3** and **f4** are printed. All these keywords allow to modify the extension of the left and right wings.

- **res**: this keyword allows to modify the resolution at which the synthetic spectra are convolved. If the new resolution differs from the old one by more than 50%

(ratio new/old outside 0.5–1.5) a confirmation is asked; if you do not confirm, the old resolution is kept.

- **dw:** this keyword allows to delete a continuum region. Once the region is deleted, the spectrum is normalized again. This keyword works only when NORM = 2 is passed.
- **aw**: this keyword allows to add a new region of continuum. Once the limits of the region are given, the observed spectrum is normalized again. This keyword works only when NORM = 2 is passed.
- **i**: this keyword allows to restore the initial values, thus erasing any action done on the observed spectrum.
- **e**: this keyword allows to end the code.

The same options are available for the **ROTAT** lines (except **f**): after each change the rotational velocity and the abundance are determined again, and the values saved with **y** are those of the last fit.

If the errors in the atmospheric parameters are provided (Sect. 3.2), the options are displayed also for each additional run with the varied parameters: in this case type **y** to accept the fit. In interactive mode the lines are always analyzed one after the other: the keyword **NPROC** is used only in automatic mode.

7. GRID OPTION

You can run `salvador` also passing a grid of precalculated synthetic spectra. To enable this option, in the **input.salvador** file you have to pass the complete path of the directory where your synthetic spectra are present. At the end of the path also a filename must be provided. This filename lists all the synthetic you want to use (see Fig. 19).

The synthetic can come from any code of spectral synthesis you want, but they must contain the wavelength and the normalized flux in the first and second column, respectively. They also have to be named in the following way:

amXXXtXXXXgXXXkXXX_YY_apXX_ELEM_YXXX_BLABLA_λ1_λ2.nb

```
am154t4248g077k195_cl_ap04_Ti_m020_7485.578_7493.578.nb
am154t4248g077k195_cl_ap04_Ti_m040_7485.578_7493.578.nb
am154t4248g077k195_cl_ap04_Ti_p000_7485.578_7493.578.nb
am154t4248g077k195_cl_ap04_Ti_p020_7485.578_7493.578.nb
am154t4248g077k195_cl_ap04_Ti_p040_7485.578_7493.578.nb
am154t4348g077k195_cl_ap04_Ti_p009_7485.578_7493.578.nb
am154t4348g077k195_cl_ap04_Ti_p029_7485.578_7493.578.nb
am154t4348g077k195_cl_ap04_Ti_p049_7485.578_7493.578.nb
```

Figure 19. Example for the Ti of how the filename of synthetic spectra should appear.

where:

- The first two letters could be **am** or **ap**. **m** stands for −, while **p** for +. This keyword codes the sign of the metallicity.
- After **am** or **ap**, the **XXX** represent the string associated with the metallicity value.
- After the **t**, the **XXXX** represent the string associated with the temperature value.
- After the **g**, the **XXX** represent the string associated with the logg value.
- After the **k**, the **XXX** represent the string associated with the ξ value.
- The **YY** separated by _ must be present.
- After the **YY**, there could be **ap** or **am** followed by **XX**. The letter **p** or **m** sets the sign of the α enhancement/depletion, while the **XX** sets the value of this enhancement/depletion.
- **ELEM** defines the name of the element for which variation is present in the synthetic.
- **YXXX** can be coded in this way: **Y** can be **m** or **p**, setting the sign (− or +) of the variation in abundance of **ELEM**. **XXX** represent the string associated with the abundance variation of the element defined in **ELEM**.
- $\lambda 1$ and $\lambda 2$ define the range covered by the synthetic spectrum. If the line lies on that range it will be analyzed, otherwise it will be discarded.
- To the date the file must end with the extension **.nb**.

Example:

am140t4502g168k174_cl_ap04_Al_p030_5500_9000.nb

This is the name of a synthetic spectrum with metallicity [M/H] = −1.40 dex, $T_{\rm eff}$ = 4502 K, $\log g$ = 1.68 dex, ξ = 1.74 km s^{-1}, with an enhancement in α of +0.40 dex, and with a variation in Al of +0.30 dex with respect to the abundance of the model. In this case is [Al/Fe] = +0.30 dex. Moreover, the spectrum covers the range between 5500 and 9000 Å.

The names of all the synthetic in the synthetic filename must have this syntax. Also, the synthetic spectra should be NOT broadened, since `salvador` does a convolution with a Gaussian function.

A limitation of the precomputed grid is the fact that if for a given element the abundance of the line analyzed is higher/lower than that present in the range of the synthetic, an incorrect value of abundance will be found since no further calculations of synthetic are performed. Also the syntax of the names is quite strict to the date.

8. ROTATIONAL VELOCITY

With the keyword **ROTAT** in the first column of the linelist (see Sect. 3.2.2), `salvador` measures from a line the projected rotational velocity of the star together with the abundance of the element. The synthetic spectra are calculated by `K2`: this option is not available with a grid of precomputed synthetic spectra. The procedure is the following:

- A grid of abundances is defined around the initial value (eighth column of the linelist, or the abundance of the model atmosphere plus the seventh column), in steps of 0.1 dex within ± the range given in the ninth column of the linelist (or by the keyword **ROT_ABU_SPAN**). With a range equal to 0 the abundance is kept fixed. For each abundance a synthetic spectrum is calculated by `K2` and convolved with a Gaussian function at the resolution of the line.

- Each of these spectra is broadened with a rotational profile (`PyAstronomy` rotBroad[10], with a limb-darkening coefficient of 0.6) for rotational velocities from the minimum to the maximum value (tenth and eleventh columns of the linelist, or keywords **ROT_MIN** and **ROT_MAX**) in steps given by the keyword **ROT_STEP** (default 10 km s^{-1}; the maximum value is always included). Since the rotation is applied to the spectra already calculated, a finer step of the rotational velocity does not require more synthetic spectra from `K2`.

- The χ^2 between the observed line and each spectrum of this grid (abundance × rotational velocity) is calculated in the region between the limits of the line (fifth and sixth columns of the linelist), as for the abundances. Around the minimum a finer grid is created by interpolation, in steps of 1 km s^{-1} within ±20 km s^{-1} and of 0.01 dex within ±0.20 dex (cubic interpolation when there are at least 4 values for each quantity, otherwise linear). The minimum of the χ^2 on this grid gives the rotational velocity and the abundance.

- The errors are estimated with a Monte Carlo simulation: noise, with the dispersion of the residuals between the observed spectrum and the best fit, is added to the best fit and the minimum is searched again on the fine grid. This is repeated **ROT_NMC** times (default 100) and the errors are the standard deviations of the values found. As for the abundances, the noise is generated from a seed and the results are reproducible.

The synthetic spectra are calculated with an abundance variation measured with respect to the abundance of the model atmosphere, so the abundances of the grid (and the result) are absolute values A(X).

[10] https://pyastronomy.readthedocs.io/en/latest/pyaslDoc/aslDoc/rotBroad.html

In the plot of the line in the .pdf file, the main panel shows the best fit and two synthetic spectra with a rotational velocity ±10 km s^{-1} from the best value. The bottom right panel shows the χ^2 as a function of abundance and rotational velocity (contours), with the best fit and its Monte Carlo errors, or the χ^2 as a function of the rotational velocity if the abundance is fixed.

The results are saved in the ROOT_NAME_rot.fit file (see Sect. 4.2). For the **ROTAT** lines the errors due to the atmospheric parameters are not calculated.

In the testing phase (simulated spectra with known rotational velocity between 5 and 55 km s^{-1}) the rotational velocities were recovered within about 1 km s^{-1}, with a step **ROT_STEP** of 10 km s^{-1}. With a step of 20 km s^{-1} the results were biased by a few km s^{-1}, in particular when the abundance is not fixed. Remember that the limits of the region of the line must include the whole broadened line (see Sect. 3.2.2).

9. PARALLEL ANALYSIS

In automatic mode (**INTER = n**) `salvador` can analyze several lines at the same time, each on a different core of your computer. The number of lines analyzed at the same time is set by the keyword **NPROC** in the **input.salvador** file (for instance **NPROC = 6**). It cannot exceed the number of cores of your computer (on Linux you can see it with the command **nproc**): it is a good choice to leave one core free.

Each line, together with its additional runs for the errors due to the atmospheric parameters, is analyzed by a separate process in its own temporary directory (**salvador_work/worker_N**), because `K2` writes its temporary files with fixed names in the working directory. Before starting, the model atmospheres needed by all the lines are calculated once. The results are written in the same order of the linelist, so the output files are identical to those obtained with **NPROC = 1**.

On the terminal one row is printed for each line analyzed, with its result. The detailed output of each process is saved in the files **salvador_work/worker_N.log**, which are kept until the following run. In interactive mode the lines are always analyzed one after the other.

The speed-up is close to the number of processes used, but it depends on the slowest lines (usually those with all the additional runs for the errors).

10. CHECKING THE RESULTS AND TROUBLESHOOTING

10.1. *regression_check.py*

This script compares the results of a run with those of a previous run, saved as a reference. It is useful to verify that the results do not change after an update of `salvador`, a new installation, or a change in the setup. It compares the files **abundance_final.txt**, **abundance_final_isor.txt**, and all the **.fit** files of the working directory, row by row and value by value. Run it in the working directory:

- **regression_check.py save**: after a run whose results you trust, saves them as the reference (in the directory **regression_ref**).
- **regression_check.py**: after a new run, compares the results with the reference. It prints OK for each identical file, and for each difference the file, the row (spectrum and line), the column, and the two values. At the end it prints ALL RESULTS IDENTICAL or SOME RESULTS DIFFER.
- **regression_check.py --tol 0.01**: as above, but the numbers can differ by up to the given value.

10.2. *check_k2_speedup.py*

This script checks that the faster calls to `K2` described in Sect. 5.1 give the same synthetic spectra and EWs as the original calls. Run it in the working directory:

check_k2_speedup.py 5

For the first 5 lines (or the number that you give) of the first spectrum of the **lista.stars** file, it calculates each synthetic spectrum and EW in both ways, compares them value by value, and prints the time needed. It ends with ALL SPECTRA AND EWs IDENTICAL if everything is correct.

10.3. *Messages and warnings*

- Errors in the input files (a missing or invalid keyword in **input.salvador**, see Sect. 3.1) stop `salvador` with a message that explains the problem. Warnings are printed for the keywords not recognized, for the rows of **lista.stars** with incomplete errors on the atmospheric parameters, and for the rows of the linelist with an unexpected number of columns.
- If `K2` does not produce a synthetic spectrum, `salvador` stops with a message that indicates which spectrum is missing, where to find the messages of `K2`, and the most common causes (a wrong path in **linelist.xkur**, a missing model atmosphere, the `Kurucz` executables not found).
- The warnings of `Python` (for instance a division by zero or a fit that did not converge) are shown once, in a short line that indicates the type of warning, the file and the line of code where it was produced. Only some known harmless warnings are hidden. If a warning appears often and you know it is harmless, you can hide it by adding the beginning of its message to the list **HARMLESS_WARNINGS** at the beginning of the file **salvador_lib/config.py**.

11. TO DO LIST

11.1. *Convolution with NON-Gaussian profiles*

REFERENCES

Asplund, M., Grevesse, N., Sauval, A. J., & Scott, P. 2009, ARA&A, 47, 481, doi: 10.1146/annurev.astro.46.060407.145222

Blanco-Cuaresma, S., Soubiran, C., Heiter, U., & Jofré, P. 2014, A&A, 569, A111, doi: 10.1051/0004-6361/201423945

Caffau, E., Ludwig, H. G., Steffen, M., Freytag, B., & Bonifacio, P. 2011, SoPh, 268, 255, doi: 10.1007/s11207-010-9541-4

Grevesse, N., & Sauval, A. J. 1998, SSRv, 85, 161, doi: 10.1023/A:1005161325181

Kurucz, R. L. 2005, Memorie della Societa Astronomica Italiana Supplementi, 8, 14

Lodders, K. 2010, in Astrophysics and Space Science Proceedings, Vol. 16, Principles and Perspectives in Cosmochemistry, ed. A. Goswami & B. E. Reddy, 379, doi: 10.1007/978-3-642-10352-0_8

Magg, E., Bergemann, M., Serenelli, A., et al. 2022, A&A, 661, A140, doi: 10.1051/0004-6361/202142971

Magrini, L., Randich, S., Friel, E., et al. 2013, A&A, 558, A38, doi: 10.1051/0004-6361/201321844

Sbordone, L., Caffau, E., Bonifacio, P., & Duffau, S. 2014, A&A, 564, A109, doi: 10.1051/0004-6361/201322430

APPENDIX

A. COWS.PY

cows.py is a simple `Python` script that allows to create the continuum regions file requested by `salvador`. **cows** stands for continuum window selector. To execute **cows.py** you need:

- An observed spectrum (either normalized or not) from which select the continuum regions.
- A synthetic spectrum that will be overimposed to the observed one. To the date the synthetic spectrum must contain 4 columns, in the following order: first: the wavelength expressed in Å, second: the normalized flux, third: the line flux, fourth: the continuum flux.
- An input file named **input_cows.txt** in which you have to specify some parameters[11].

The **input_cows.txt** is structured as follows:

- First row: contains the complete path in which the synthetic spectrum is located.
- Second row: contains the name of the observed spectrum. It can be either in the FITS or ASCII extension.
- Third row: contains a keyword (**y** or **n**) that allows to degrade the resolution of the synthetic spectrum or to keep it as is.
- Fourth row: contains the resolution at which the synthetic spectrum will be convolved. This value is read only if the third row the keyword is set to **y**.
- Fifth row: contains a keyword (**y** or **n**) that allows to correct the observed spectrum for the RV provided in input or to estimate the RV from cross-correlation[12] between the observed and the synthetic spectrum.
- Sixth row: contains the value of RV that will be applied to the observed spectrum. This value will be used only if in the fifth row the keyword is set to **y**.
- Seventh and eighth rows: contain the range in which the continuum regions will be selected.

In Fig. 20 is displayed the structure of the input file.

[11] The **input_cows.txt** is present in the **tutotial** subdirectory.
[12] https://pyastronomy.readthedocs.io/en/latest/pyaslDoc/aslDoc/crosscorr.html

```
path_syn          = /path/to/synthetic/spectrum/synth_name
name_spec         = observed_spectrum
degrade           = y
resol             = 20000
est_rv            = n
RV                = 130.0
l1                = start_wave
l2                = end_wave
```

Figure 20. Example of the structure of **input_cows.txt**.

Once the input file is set, you can run the code from terminal by typing **cows.py** from the directory in which the observed spectrum is located.

As a first step the code will show you the observed spectrum with overimposed the continuum of the spectrum, that is calculated by using a median filter on the flux through the `scipy.signal` medfilt function. On the terminal will be asked you if the kernel size used is okay (**y**) or if you want to change the size and recompute the continuum shape (**k**). Once you are satisfied the normalized spectrum will be displayed and then closed automatically. At this point the normalized (and shifted) and the synthetic spectra will be displayed together and you can start the selection of continuum regions. To do that you can click with the left button of the mouse on a region of the spectrum that has a continuum. This represents the left edge of that continuum portion of spectrum. Then you click again with the left button on another region that represents the right edge of that given continuum region (see Fig. 21). On the portion of the plot that is displayed you can select as many regions as you can. If you are not satisfied with the **LAST** selected point you can unselect it by clicking on it with the right button of the mouse.

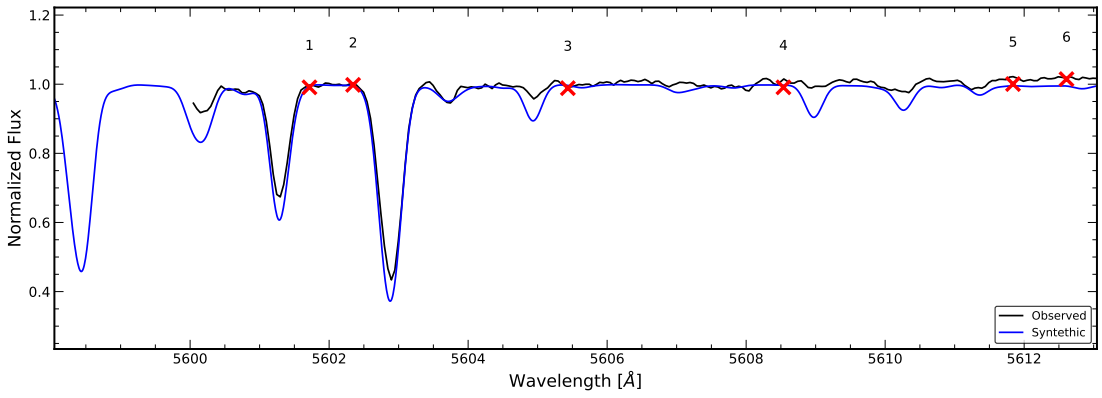


Figure 21. Example of how the selection of continuum regions works.

The plot displays a window of 15 Å. When you select all the continuum regions on that window you can close the plot and on the terminal will be asked if you want to finish the code (**y**) or not (**n**). If you finish the code, a file named **wind_continuum.txt** will be created with N rows and 2 columns per row, each row representing a continuum

portion of the spectrum. If you do not finish the code you can continue select continuum regions and a new portion of the spectra will be displayed with the following 15 Å.

You can change the default window size of 15 Å, by modifying the keyword **step_plot** directly into the **cows.py** script.

B. MERGE_FIT.PY

merge_fit.py is a `Python` script that allows to merge different **.fit** files of the same star, in order to create a unique fit file for that star. Moreover, it creates a new file, named **abundance_final_merged.txt** that is the corresponding of the **abundance_final.txt**, but where the final abundances are calculated using the unique **.fit** file just created.

You can use the script, running from terminal the command: **merge_fit.py**.

At this point (see Fig. 22), you need to provide the name of a file containing the **.fit** files that you want to merge in a unique file. The file must be organized in the following way: in each row you provide each name of the **.fit** files you want to merge, in the last row you provide the output name for the merged **.fit** file (example, star1.fit). Then to move to other files to merge, you separate them from the previous ones with ###. Finally, you can decide to use the [Fe/H] abundance directly from the **.fit** files, or to use an external value. In this latter case, after the output name you have to provide the [Fe/H] value that will be used to scale the abundance ratios. See Fig. 23 for a better visual explanation.

Then you need to provide the solar reference you want to use to scale abundance ratios (they are coded with a number). Finally, you need to provide a value (0 or 1) in which you specify if you want to use the original flags present in the single **.fit** files (1) or if you want to recalculate the flags (0). If you provide 0, then you have to provide the value for the σ-clipping, the EP_min, EP_max, EWR_min, and EWR_max. These values will be used to calculate the final abundances.

```
Filename containing the .fit files to merge: ciao.txt

Which solar reference? (1 -> CL | 2 -> GS98 | 3 -> Magg22 | 4 -> Asp09 ):  1

Recalculate the flags (0) or use the flags present in the .fit files (1)?:  0


--- Parameters for sigma, EWR, and EP clipping ---
Insert value for sigma clipping? 3
Insert EP min (es. 0.0): 0.
Insert EP max (es. 10.0): 8.5
Insert EWR min (es. -6.0): -6.5
Insert EWR max (es. -4.0): -4.5
```

Figure 22. Example of how to use **merge_fit.py** once you ran the script. If you provide 1 in the answer for the flags, the five questions about the parameters will not appear,

```
star1_1.fit
star1_2.fit
star1.fit ^
###
star2_1.fit
star2_2.fit
star2_3.fit
star2.fit       -1.60
###
```

Figure 23. Example of how the input_file for **merge_fit.py** should be organized. In this specific case, 2 new **.fit** files will be created, **star1.fit** and **star2.fit**. For the second file [Fe/H] = −1.60 will be used to scale all the abundance ratios, as it is provided by the user, while for the first file [Fe/H] will be determined from the Fe lines present.

C. STRUCTURE OF THE CODE

This appendix is useful only if you want to read or modify the code. The program that you run is **salvador.py**: it reads the input files, analyzes the lines of each spectrum (functions analyse_line and write_line_result), and writes the final files. All the other functions are in the directory **salvador_lib**, divided into modules according to their task (Table 1). The file **utilities_salvador.py** makes all of them available with a single command (*from utilities_salvador import **). Each function starts with a short description of what it does, and the file **FUNCTION_MAP.txt** lists all the functions with the module where each one is.

Table 1. Modules of the directory **salvador_lib**.

Module	Content
config.py	settings of the run (keywords of input.salvador), warnings, constants, paths
elements.py	solar abundances, symbols of the elements, abundances of the model atmosphere
k2_interface.py	everything that uses `K2`: synthetic spectra, EWs, reduced linelists
spectra_io.py	reading of the observed spectra and of the grids of synthetic spectra
fit_files.py	the .fit files: errors, flags, reading
normalization.py	normalization of the spectra (NORM) and continuum regions
fitting.py	resolution, convolution, abundance of a line (χ^2), Monte Carlo
rotation.py	rotational velocity of a line
plotting.py	plots of the lines
summary.py	final abundances: mean, σ-clipping, errors
line_analysis.py	analysis of one line (measure_line): first fit, interactive menu, automatic mode
parallel.py	parallel analysis of the lines (NPROC)

All the settings of the **input.salvador** file are collected in a single object, *cfg*, and in the code they are always written as *cfg.name* (for instance *cfg.window* or

cfg.type_norm). To add a new keyword, add one row in the table KEYWORDS and one field in the class Config, both at the beginning of **salvador_lib/config.py**. After any change to the code you can use **regression_check.py** to verify that the results did not change (Sect. 10).